

LAB EXPERIMENTS

11 .Perform a 180-degree rotation clockwise along the y-axis for the given image

A)Program:-

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/L-IMAGE.jpg')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Rotate image
rotated = cv2.rotate(img_rgb, cv2.ROTATE_180)

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(rotated)
plt.title("180 Degree Rotated")
plt.axis("off")

plt.show()
```

Output:

Original Image



180 Degree Rotated



12.Perform a 270-degree rotation clockwise along the y-axis for the given image.

A)Program:-

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/L-IMAGE.jpg')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Rotate image
rotated = cv2.rotate(img_rgb, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imwrite("/content/rotated-270.jpg", rotated)

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(rotated)
plt.title("270 Degree Rotated")
plt.axis("off")

plt.show()
```

Output:

Original Image



270 Degree Rotated



13.Perform Affine Transformation on the given image using python and Open CV

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/sf.webp')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

rows, cols = img.shape[:2]

# Define points
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])

# Affine transformation
```

```

matrix = cv2.getAffineTransform(pts1, pts2)
output = cv2.warpAffine(img_rgb, matrix, (cols, rows))

cv2.imwrite("/content/Affine.jpg", output)

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(output)
plt.title("Affine Transformation")
plt.axis("off")

plt.show()

```

Output:

Original Image



Affine Transformation



14. Perform Perspective Transformation on the given image using python and Open CV.

A) Program:

```

import cv2
import numpy as np

```

```
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/sloth.avif')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

rows, cols = img.shape[:2]

# Define points
pts1 = np.float32([[50,50],[200,50],[50,200],[200,200]])
pts2 = np.float32([[10,100],[200,50],[100,250],[220,220]])

# Perspective transformation
matrix = cv2.getPerspectiveTransform(pts1, pts2)
output = cv2.warpPerspective(img_rgb, matrix, (cols, rows))
cv2.imwrite("/content/perspective.jpg", output)

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(output)
plt.title("Perspective Transformation")
plt.axis("off")

plt.show()
```

Output:

Original Image



Perspective Transformation



15. Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function

A) Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/W.jpg')

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect corners
gray = np.float32(gray)
corners = cv2.cornerHarris(gray, 2, 3, 0.04)

# Mark corners in red
img[corners > 0.01 * corners.max()] = [0,0,255]

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.imwrite("/content/corner.jpg", img_rgb)
```

```
# Display image
plt.imshow(img_rgb)
plt.title("Harris Corner Detection")
plt.axis("off")

plt.show()
```

Output:



Original image

Harris Corner Detection



16.Implement a Sobel algorithm using Open CV to filter the input image.

A)Program:

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/sf.webp', 0)

# Apply Sobel filter
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
cv2.imwrite("/content/sobelx.jpg", sobelx)
cv2.imwrite("/content/sobely.jpg", sobely)

# Display images
plt.figure(figsize=(10,5))

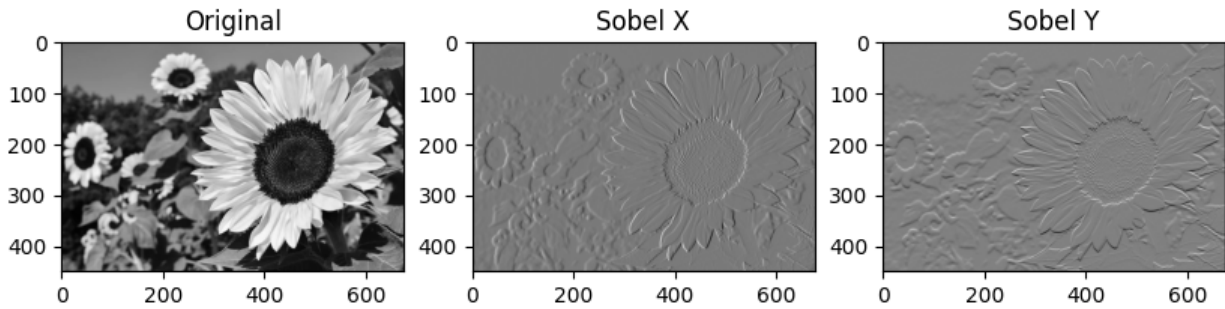
plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(sobelx, cmap='gray')
plt.title("Sobel X")

plt.subplot(1,3,3)
plt.imshow(sobely, cmap='gray')
plt.title("Sobel Y")

plt.show()
```

Output:



17.Design and implement a water marking technique to insert the watermark into the original effectively image using Open CV.

A)Program:

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/butterfly.jpg')

# Add watermark text
watermarked = cv2.putText(
    img.copy(),
    'WATERMARK',
    (50,50),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (255,0,0),
    2
)

# Convert to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
watermark_rgb = cv2.cvtColor(watermarked, cv2.COLOR_BGR2RGB)
cv2.imwrite("/content/watermarked.jpg",watermark_rgb)
# Display images
plt.figure(figsize=(10,5))

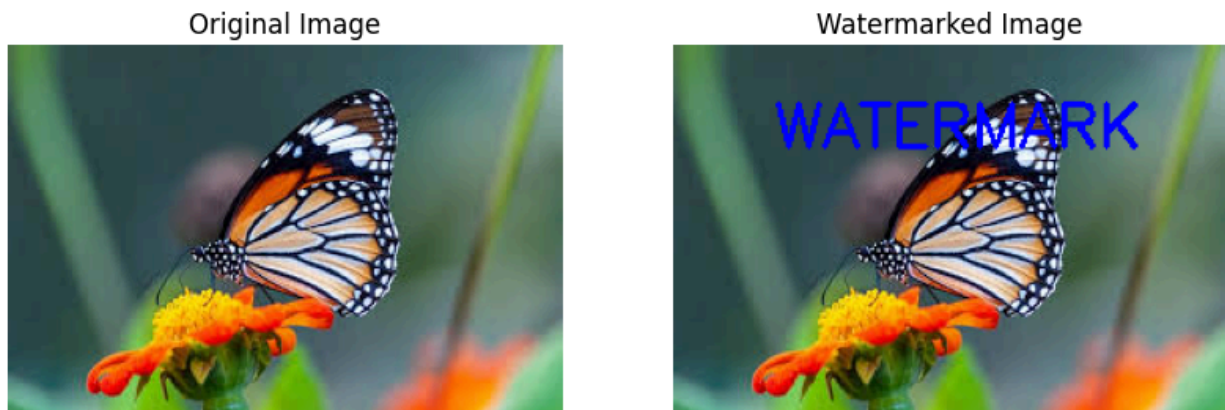
plt.subplot(1,2,1)
```

```
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(watermark_rgb)
plt.title("Watermarked Image")
plt.axis("off")

plt.show()
```

Output:



18. Implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using Open CV.

A) Program:

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/panda.jpg')

# Check image loaded or not
if img is None:
    print("Error: Image not found")
else:
    # Convert BGR to RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```

# Crop image
crop = img[50:150, 50:150]

# Get image dimensions to determine valid paste area
img_height, img_width, _ = img.shape
crop_height, crop_width, _ = crop.shape
# Define paste coordinates
# Ensure the paste area is within the image boundaries
# For example, let's paste at (10, 10) if there's enough space
paste_start_y = 10
paste_end_y = paste_start_y + crop_height
paste_start_x = 10
paste_end_x = paste_start_x + crop_width

if paste_end_y <= img_height and paste_end_x <= img_width:
    # Paste cropped part
    img[paste_start_y:paste_end_y, paste_start_x:paste_end_x] = crop
    # Display image
    plt.imshow(img)
    plt.title("Cropping Copying and Pasting")
    plt.axis("off")
    plt.show()
    cv2.imwrite("/content/cropped_pasted_panda.jpg", cv2.cvtColor(img,
cv2.COLOR_RGB2BGR))
else:
    print(f"Error: Cannot paste crop of size {crop_width}x{crop_height} at
({paste_start_x}, {paste_start_y}) into image of size {img_width}x{img_height}.")
    print("Please adjust crop dimensions or paste coordinates.")

```

Output:



Original image

Cropping Copying and Pasting



19. Implement the Erosion Morphological operations technique using Open CV in python.

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/eye.jpg')

# Create kernel
kernel = np.ones((5,5), np.uint8)

# Apply erosion
eroded = cv2.erode(img, kernel, iterations=1)

# Convert to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
eroded_rgb = cv2.cvtColor(eroded, cv2.COLOR_BGR2RGB)
cv2.imwrite("/content/erode.jpg", eroded_rgb)

# Display images
plt.figure(figsize=(10,5))

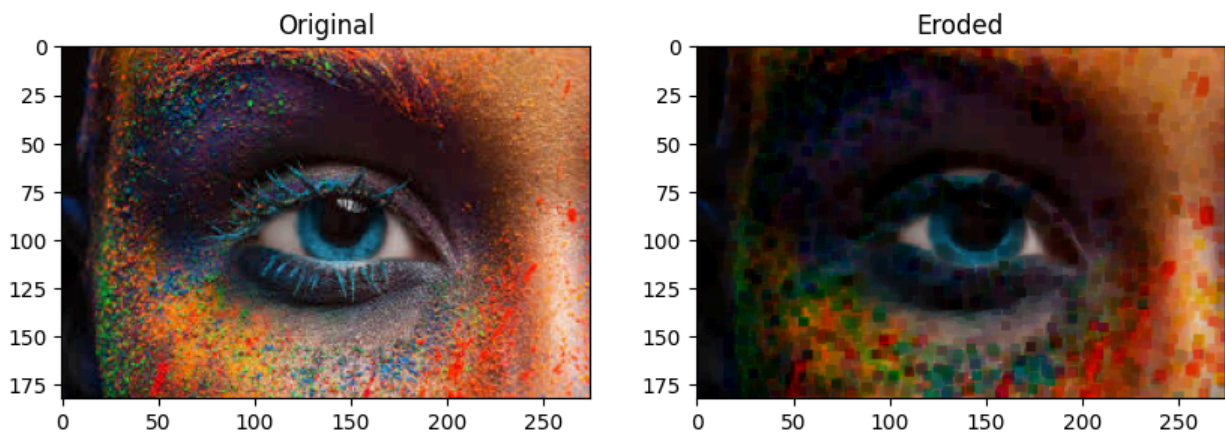
plt.subplot(1,2,1)
```

```
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(eroded_rgb)
plt.title("Eroded")

plt.show()
```

Output:



20. Implement the Dilation technique as a Morphological operation to dilate the foreground regions based on Open CV.

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/ti.jpg')

# Create kernel
kernel = np.ones((5,5), np.uint8)

# Apply dilation
dilated = cv2.dilate(img, kernel, iterations=1)
```

```

# Convert to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dilated_rgb = cv2.cvtColor(dilated, cv2.COLOR_BGR2RGB)
cv2.imwrite("/content/dilated image.jpg", dilated_rgb)
# Display images
plt.figure(figsize=(10,5))

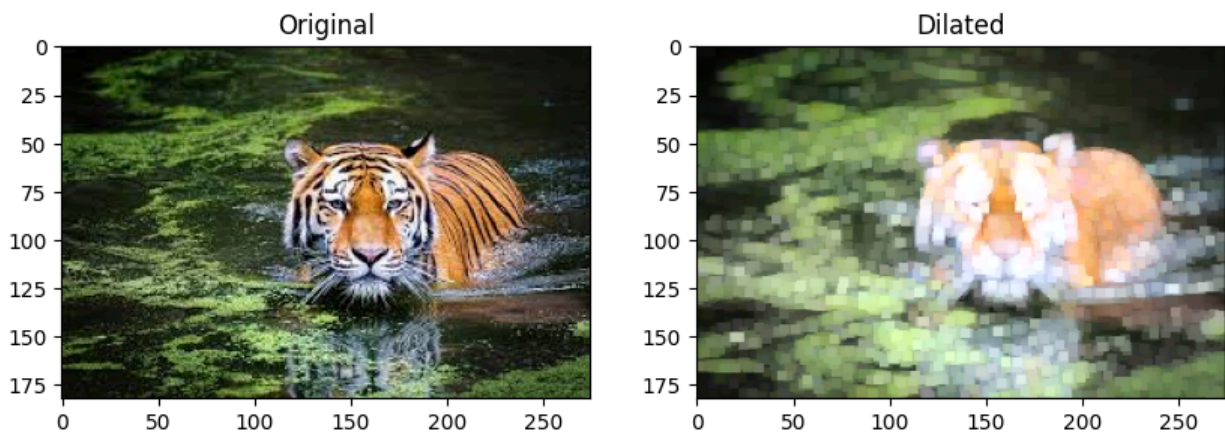
plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(dilated_rgb)
plt.title("Dilated")

plt.show()

```

Output:



21. Implement the Opening technique as a Morphological operation to dilate the foreground regions based on Open CV.

A)Program:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image

```

```

img = cv2.imread('/content/W.jpg')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Create kernel
kernel = np.ones((5,5), np.uint8)

# Apply opening
opening = cv2.morphologyEx(img_rgb, cv2.MORPH_OPEN, kernel)
cv2.imwrite("/content/morpho-opening.jpg", opening)
# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(opening)
plt.title("Opening")

plt.show()

```

Output:



22. Implement the Closing technique as a Morphological operation to dilate the foreground regions based on Open CV.

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/sloth.avif')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Create kernel
kernel = np.ones((5,5), np.uint8)

# Apply closing
closing = cv2.morphologyEx(img_rgb, cv2.MORPH_CLOSE, kernel)
cv2.imwrite("/content/morpho-closing.jpg", closing)

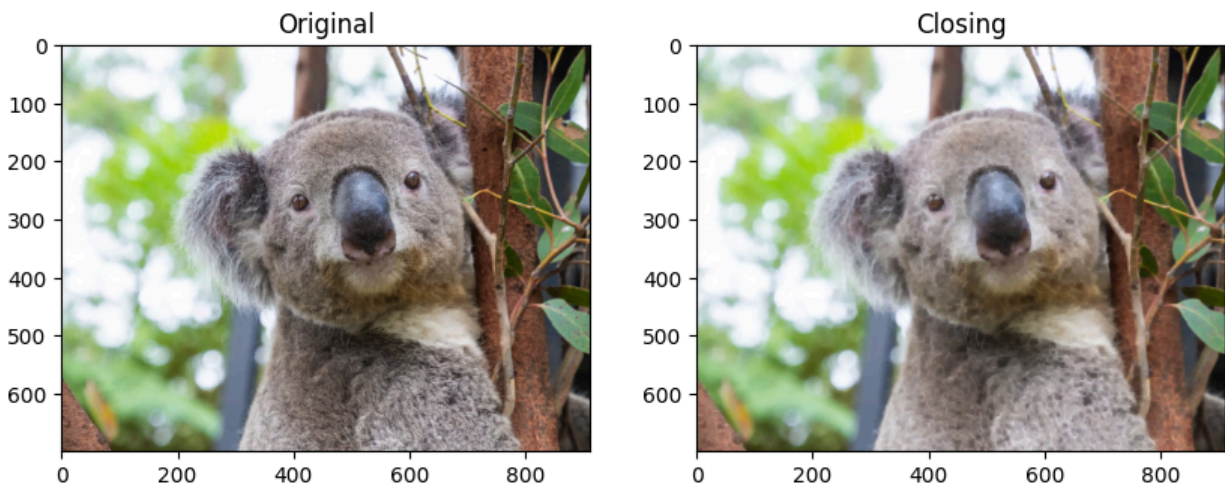
# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(closing)
plt.title("Closing")

plt.show()
```


Output:



23. Implement the Top hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/JAS.jpg')

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Create kernel
kernel = np.ones((5,5), np.uint8)

# Apply top hat
tophat = cv2.morphologyEx(img_rgb, cv2.MORPH_TOPHAT, kernel)
cv2.imwrite("/content/morpho-top hat.jpg", tophat)

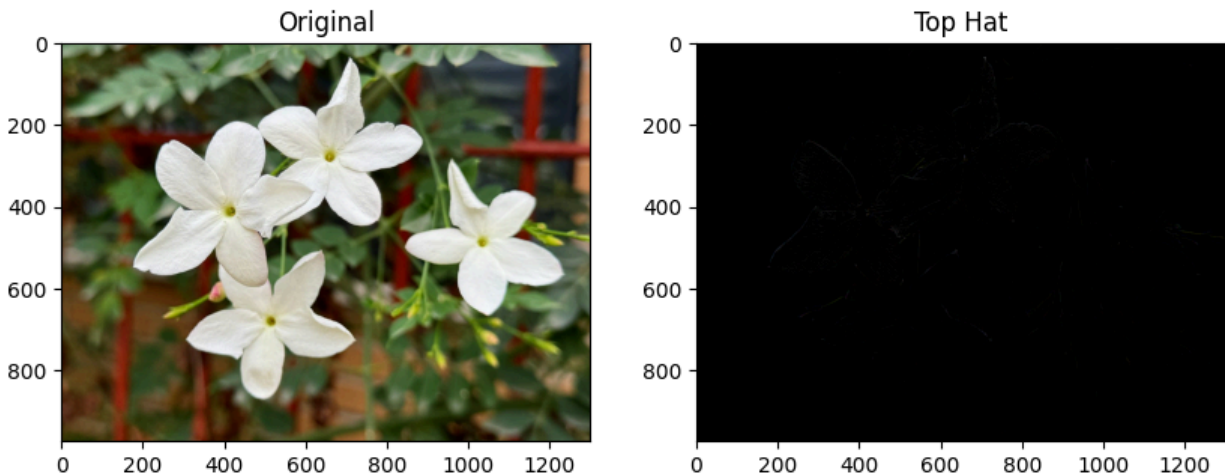
# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")
```

```
plt.subplot(1,2,2)
plt.imshow(tophat)
plt.title("Top Hat")
```

```
plt.show()
```

Output:



24. Implement the Black hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

A)Program:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('/content/JAS.jpg')

# Check if image was loaded successfully
if img is None:
    print("Error: Image not found or could not be loaded. Please check the file path.")
else:
    # Convert BGR to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Create kernel
```

```

kernel = np.ones((5,5), np.uint8)

# Apply black hat
blackhat = cv2.morphologyEx(img_rgb, cv2.MORPH_BLACKHAT, kernel)
cv2.imwrite("/content/morpho-black hat.jpg", blackhat)
# Display images
plt.figure(figsize=(10,5))

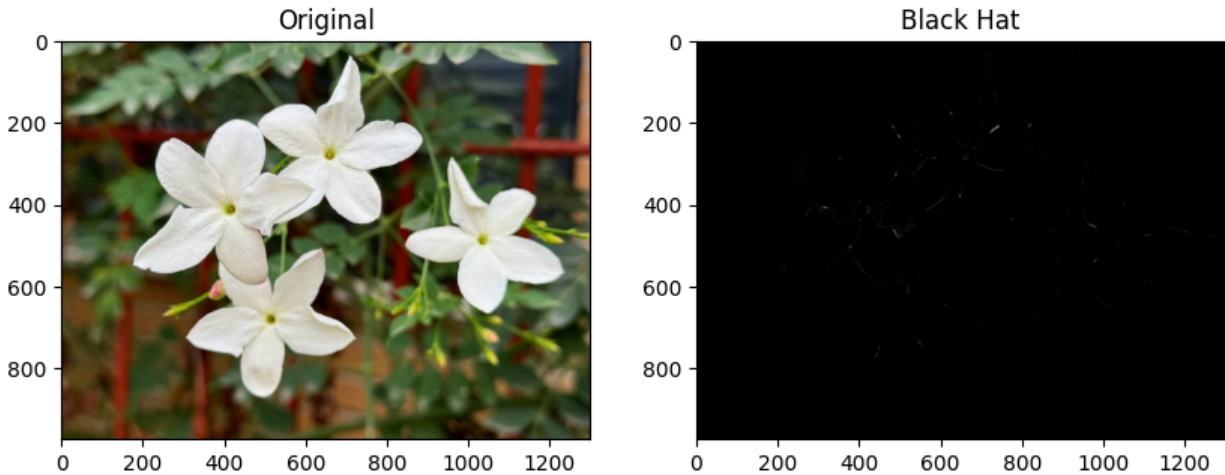
plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(blackhat)
plt.title("Black Hat")

plt.show()

```

Output:



25. Recognize watch from the given image by general Object recognition using Open CV.

A) Program:

```

import cv2
import matplotlib.pyplot as plt

```

```
# Read image
img = cv2.imread('/content/watch.webp')

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Blur image
blur = cv2.GaussianBlur(gray, (5,5), 0)

# Edge detection
edges = cv2.Canny(blur, 50, 150)

# Find contours
contours, _ = cv2.findContours(edges,
                                cv2.RETR_EXTERNAL,
                                cv2.CHAIN_APPROX_SIMPLE)

# Find largest contour
largest = max(contours, key=cv2.contourArea)

# Draw bounding box
x, y, w, h = cv2.boundingRect(largest)

cv2.rectangle(img, (x,y), (x+w, y+h), (255,0,0), 3)

# Add label
cv2.putText(img, 'Watch', (x, y-10),
            cv2.FONT_HERSHEY_SIMPLEX,
            1, (255,0,0), 2)

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.imwrite("/content/object recognition.jpg", img_rgb)

# Display output
plt.imshow(img_rgb)
plt.title("object Detection")
plt.axis("off")

plt.show()
```

Output:

Original image



object Detection

